

TensorFlow框架简单介绍

2018-04-25 zzz_CMing [Python爱好者社区](#)



点击蓝字 轻松关注



作者：zzz_CMing

博客：http://blog.csdn.net/zzz_cming

作者好文推荐：

[CNN卷积神经网络原理讲解+图片识别应用（附源码）](#)

[BP神经网络+TensorFlow做图片识别](#)

[RNN递归神经网络原理介绍+数字识别应用](#)

[【无监督学习】K-means聚类算法原理介绍，以及代码实现](#)

[【无监督学习】DBSCAN聚类算法原理介绍，以及代码实现](#)

[【无监督学习】Density Peaks聚类算法（局部密度聚类）](#)

前言：TensorFlow框架的安装以及环境配置，这里就省略了，网上可以找到许多用安装包、pip方式安装的方法，不再赘述。这里通过对TensorFlow的计算模型、数据模型以及运行模型三个角度的介绍，希望大家能对TensorFlow的工作原理能有一个大致的了解。



“TensorFlow”这个名字是个双拼的词——“Tensor” + “Flow”

- Tensor：张量。（张量这个概念是引用于数学或物理上的，而且在数学和物理上的解释也不完全相同，当然我们不是来研究它具体是什么的）。**在TensorFlow中，张量可以被简单理解为多维数组**（下面有详细解释）
- Flow：流、飞。直接表达了张量之间通过计算相互转化的过程

总结：“Tensor”表明了TensorFlow的数据结构，“Flow”体现了TensorFlow的计算模型。下面的分主题，就是依照这两个角度分别讲解。

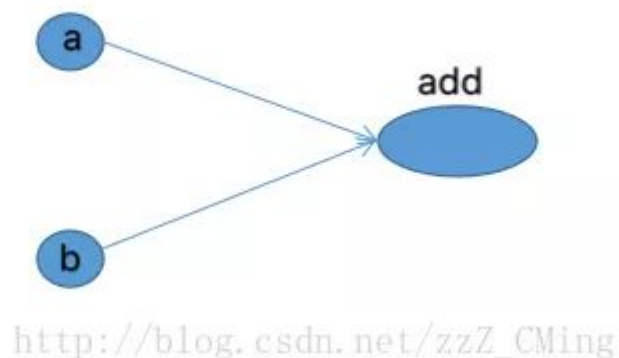
一、TensorFlow的计算模型——计算图

计算图是TensorFlow中最基本的一个概念，TensorFlow中的所有计算都会被转化为计算图上的节点，依据节点中传递的值，进行数值的“Flow”。

1、计算图的概念

TensorFlow是一个通过计算图的形式来表述计算的编程系统——TensorFlow中的每一个计算都是计算图上的一个节点，而节点之间的边描述了计算之间的依赖关系。

简单的说，在计算图里任意一个节点上，完成计算，得到输入或输出的结果；边的连接，表明了数据的走向或计算的依赖。计算图就是数据在“节点 - 边 - 节点”这种周转方式下抽象出来的图形概念。下图是通过TensorBoard画出来的两个向量相加样例的计算图



很直观的表明节点add依赖于向量a、b的输入。所有TensorFlow的程序都可以通过上图所示的计算图的形式来表示，这也就是TensorFlow的基本计算模型。

2、计算图的使用

TensorFlow程序一般可以分为两个阶段：第一阶段预定义计算图中所有的计算，不论是数据还是计算方法。第二阶段就是指定具体数值，执行计算得到结果。

以下代码给出了计算定义阶段的样例

```
import tensorflow as tf a = tf.constant([1.0,2.0],name = "a") b =  
tf.constant([2.0,3.0],name = "b") result = a + b
```

在这个过程中，TensorFlow会自动将定义的计算转化为计算图上的节点，可以通过`tf.get_default_graph()`函数获取当前默认的计算图，以下代码示意了如何获取默认计算图以及如何查看一个运算所属的计算图：

```
print(a.graph is tf.get_default_graph())
```

二、TensorFlow数据模型——张量

1、张量的概念

张量是TensorFlow管理数据的形式。在TensorFlow中，所有的数据都是通过张量的形式来表示。从功能的角度上看，张量可以简单理解为多维数组。其中

- 零阶张量表示标量，也就是一个数
- 一阶张量表示向量，也就是一个一维数组
- N阶张量可以理解为一个N维数组

但是张量在TensorFlow中的实现并不是直接采用数组的形式，它只是对TensorFlow中运算结果的引用；在张量中并没有真正保存数字，它保存的是如何得到这些数字的计算过程。例如引用下面代码，当运行时并不会得到加法的结果，而是得到对结果的一个引用：

```
import tensorflow as tf #tf.constant是一个计算，这个计算的结果为一个张量，保  
存在变量中 a = tf.constant([1.0,2.0],name = "a") b =  
tf.constant([2.0,3.0],name = "b") result = tf.add(a,b,name = "add")  
print(result) """ 输出如下： Tensor("add:0", shape=(2,), dtype=float32) """
```

从上结果可以看出TensorFlow中的张量与NumPy中的数组是不同的——TensorFlow计算的结果并不是一个具体的数字，而是一个张量结构，主要保存张量的三个属性：名字（name）、维度（shape）和类型（type）

- 名字 (name)：不仅是一个张量的唯一标识符，同样也给出了这个张量是如何计算出来的——上面已经介绍了TensorFlow的计算都可以通过计算图的模型来建立，计算图中每个节点代表一个计算，计算的结果保存在张量之中，**所以张量与计算图上节点所代表的计算结果相对应**。这样张量的命名就可以通过 “node:scr_output” 的形式给出，其中node为节点名称，scr_output代表当前张量来自节点的第几个输出。例上 “add: 0” 就说明result这个张量是节点 “add” 输出的第一个结果。
- 维度 (shape)：这个属性描述了一个张量的维度信息。比如上面样例中 shape= (2,) 说明了张量result是一个一维数组，数组长度为2
- 类型：每一个张量会有一个唯一的类型，TensorFlow会对参与运算的所有张量进行类型检查，只有类型都匹配，才能输出正确结果。TensorFlow支持14种不同的类型，这里就不一一罗列了

2、张量的使用

张量的使用主要可以总结为两大类：

- 第一类：对中间计算结果的引用。当一个计算包含很多中间结果时，使用张量可以很大程度提高代码的可读性
- 第二类：当计算图构造完成，用张量来获得计算结果，也就是真实值。虽然张量本身没有存储具体的数字，但是可以通过下面介绍的会话 (session) 就可以得到这些具体的数字。

三、TensorFlow运行模型——会话

前面介绍了TensorFlow是如何组织数据和运算的，本节将介绍如何使用会话 (session) 来执行定义好的运算。

第一种，需要明确调用会话的生成函数和关闭函数，防止错误运行时候导致的资源泄露。

代码流程如下：

```
#创建一个会话 sess = tf.Session() #调用需要执行的语句；例如sess.run(result)
sess.run(...) #关闭会话 sess.close()
```

第二种，通过Python的上下文管理器来使用会话，不用担心异常退出时资源释放的问题。

代码流程如下：

```
#创建一个会话,用上下文管理器来管理会话 with tf.Session() as sess:  
    sess.run(...) #不需要调用Session.close()函数关闭会话
```

温馨提示：

- sess.run()函数是执行函数
- 交互环境下要用tf.InteractiveSession()函数直接构建默认会话

重要提示：

在具体使用TensorFlow框架处理问题的程序中，下面这段语句要放在主程序的前面

```
sess = tf.InteractiveSession() init = tf.global_variables_initializer()  
sess.run(init)
```

- **上面是在具体使用TensorFlow框架前，激活TensorFlow框架，初始化变量的语句**

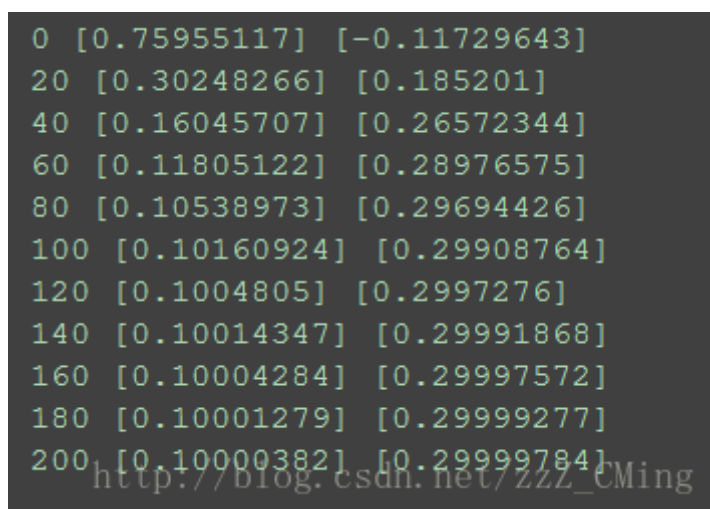
四、举例TensorFlow框架做直线拟合

```
# -*- coding:utf-8 -*- # -*- author: zzZ_C Ming # -*- 2018/01/22;22:13 # -*-  
python3.5 import tensorflow as tf import numpy as np import os  
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2' #初始化：随机生成输入数据X，得到对应  
真实值Y X = np.random.rand(100).astype(np.float32) Y = X*0.1+0.3 """ ###  
tensorflow框架——开始 """ #用随机数列生成的方式，得到结构是1维，范围在[-1,1]  
的权重 W = tf.Variable(tf.random_uniform([1],-1.0,1.0)) #用随机数列生成的方  
式，得到全0的偏置值 b = tf.Variable(tf.zeros([1])) #预测值y y = W*X+b #计算预  
测值与真实值的误差 loss = tf.reduce_mean(tf.square(y - Y)) #梯度下降法：选择  
GradientDescentOptimizer优化器，学习率为0.5 train =
```

```
tf.train.GradientDescentOptimizer(0.5).minimize(loss) #tf框架：初始化变量
init = tf.global_variables_initializer() """ ### tensorflow框架——结束 """ #
激活init sess = tf.Session() sess.run(init) for i in range(201):
    sess.run(train)      if i%20 == 0:
    print(i, sess.run(W), sess.run(b))
```

主要看TensorFlow框架如何定义、初始化；sess.run()函数如何运行

结果展示：



```
0 [0.75955117] [-0.11729643]
20 [0.30248266] [0.185201]
40 [0.16045707] [0.26572344]
60 [0.11805122] [0.28976575]
80 [0.10538973] [0.29694426]
100 [0.10160924] [0.29908764]
120 [0.1004805] [0.2997276]
140 [0.10014347] [0.29991868]
160 [0.10004284] [0.29997572]
180 [0.10001279] [0.29999277]
200 [0.10000382] [0.29999784]
```

http://blog.csdn.net/zzz_cMing

结束语：后续还有TensorFlow各种变量的初始化、TensorBoard可视化、以及用TensorFlow完成应用的相关文章，请大家关注。

[赞赏作者](#)



“谢谢你的支持”

长长长长长明 的赞赏码



Python爱好者社区历史文章大合集：

[Python爱好者社区历史文章列表（每周append更新一次）](#)



福利：文末扫码立刻关注公众号，“Python爱好者社区”，开始学习Python课程：

关注后在公众号内回复“课程”即可获取：

小编的Python入门视频课程！！！！

崔老师爬虫实战案例免费学习视频。

丘老师[数据科学入门指导](#)免费学习视频。

陈老师[数据分析报告制作](#)免费学习视频。

[玩转大数据分析！Spark2. X+Python 精华实战课程](#)免费学习视频。

丘老师[Python网络爬虫实战](#)免费学习视频。

